

Topology Optimization

Lecture 3 – FEM 2

Poisson's equation in FEniCS

Linear Elasticity

Newton's method

Non-linear Poisson's equation

Non-linear Elasticity

Course overview

No.	Topics
1	Introduction Function and vector spaces, Interpolation, L^2 projection
2	The Finite Element Method for PDEs 1: Poisson's Equation, The Finite Element Method, Properties of the Finite Element Method
3	The Finite Element Method for PDEs 2: Index notation, Linear Elasticity, Non-linear Poisson's equation, Non-linear Elasticity
4	Constrained Optimization 1: Optimality, KKT Theorem, Numerical Optimization Algorithms
5	Constrained Optimization 2: Method of Moving Asymptotes, MPI and PETSc
6	Introduction to Topology Optimization: The idea, Filtering and Projection, Python implementation
7	Gradients
8	Robust topology optimization, More on MPI
9	Wave equations
10	Acoustic waves and more on Robust Optimization
11	Penalty and Augmented Lagrangian Methods Stress-constraints in Topology Optimization 1
12	Stress-constraints in Topology Optimization 2
13	Light concentration using nanoparticle design
14	Heat conduction

Some mathematical prerequisites ✓ **DONE!**

Mathematical and numerical
formulation of the
Finite Element Method

Optimization Theory and
numerical optimization methods

Basic Topology Optimization

Advanced Topology Optimization

Applications of Topology Optimization

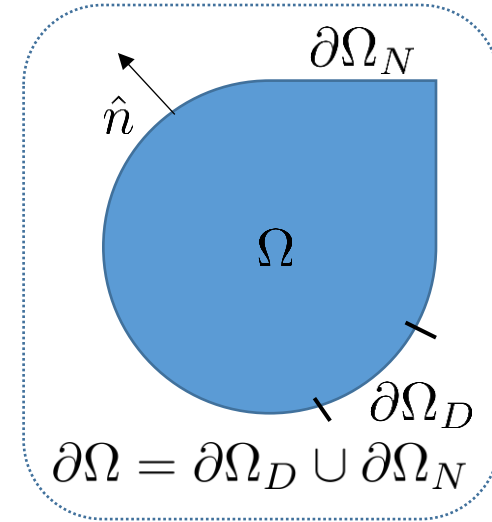
Poisson's Equation

Classical (strong) form:

$$\begin{aligned} -\nabla^2 u &= f && \text{in } \Omega && , \\ u &= u_0 && \text{on } \partial\Omega_D \subset \partial\Omega && , \\ \nabla u \cdot \hat{n} &= g && \text{on } \partial\Omega_N \subset \partial\Omega && , \quad \partial\Omega_N = \partial\Omega \setminus \partial\Omega_D \end{aligned}$$

Weak form (or variational form): Find $u \in T$ such that

$$\int_{\Omega} \nabla u \cdot \nabla v \, dx = \int_{\partial\Omega_N} g v \, dS + \int_{\Omega} v f \, dx \quad , \quad \forall v \in \hat{T}$$



Note that Neumann boundary conditions appear directly in the weak equation. The Dirichlet boundary conditions appear in the function space T and v is required to be zero on $\partial\Omega_D$.

To proceed we:

1. Choose a discrete function spaces for T and $\hat{T}$. Often $T \subset V$ and $\hat{T} \subset V$, named Galerkin FEM.
2. Expand the solution, u , in the basis functions of the space.
3. Demand the weak form is satisfied for all test functions, v , by demanding it is satisfied for all basis functions.
4. This leads to a system of linear equations that we can solve to obtain the "best approximation" (in the L^2 error) possible in the chosen function space.

We will now see how to do this in fenics with a few lines of code.

Poisson's Equation in fenics (ft01_poisson.py)

$-\nabla^2 u = f$ in the Unit Square
 $u = u_D$ on the whole boundary



Find $u \in T$ such that

$$\int_{\Omega} \nabla u \cdot \nabla v \, dx = \int_{\Omega} v f \, dx, \quad \forall v \in \hat{T}$$

$$T = \{u \in V \mid u = u_D \text{ on } \partial\Omega\}$$

$$\hat{T} = \{v \in V \mid v = 0 \text{ on } \partial\Omega\}$$

```
from fenics import *
import matplotlib.pyplot as plt
```

```
# Create mesh and define function space
mesh = UnitSquareMesh(8, 8)
V = FunctionSpace(mesh, 'P', 1)
```

```
# Define boundary condition. u_D is also exact solution.
# C++ syntax is used in 'Expression'.
u_D = Expression('1 + x[0]*x[0] + 2*x[1]*x[1]', degree=2)
```

```
def boundary(x, on_boundary):
    return on_boundary
bc = DirichletBC(V, u_D, boundary)
```

```
# Define variational problem
u = TrialFunction(V)
v = TestFunction(V)
f = Constant(-6.0)
a = dot(grad(u), grad(v))*dx
L = f*v*dx
```

```
# Compute solution
u = Function(V)
solve(a == L, u, bc)
```

```
# Plot solution and mesh
```

```
p=plot(u)
plot(mesh)
plt.colorbar(p)
```

```
# Save solution to file in VTK format
vtkfile = File('poisson/solution.pvd')
vtkfile << u
```

```
# Compute error in L2 norm
error_L2 = errornorm(u_D, u, 'L2')
```

```
# Compute maximum error at vertices
vertex_values_u_D = u_D.compute_vertex_values(mesh)
vertex_values_u = u.compute_vertex_values(mesh)
import numpy as np
error_max = np.max(np.abs(vertex_values_u_D - vertex_values_u))
```

```
# Print errors
print('error_L2 =', error_L2)
print('error_max =', error_max)
```

```
# Hold plot
plt.show()
```

Poisson's Equation in fenics (ft01_poisson.py)

```
sma@D43142:~/METOPOPT/BookExamples$ python3 ft01_poisson.py
```

Solving linear variational problem.

*** Warning: Degree of exact solution may be inadequate for accurate result in error norm.

```
error_L2 = 0.008235098073354943
```

```
error_max = 1.3322676295501878e-15
```

$$\begin{aligned} L^2 \text{ norm:} \\ ||u||_{L^2} &= \sqrt{\int_{\Omega} |u|^2 dV} \\ L^2 \text{ error:} \\ ||e||_{L^2} &= \sqrt{\int_{\Omega} |u - u_{exact}|^2 dV} \end{aligned}$$

The problem was made such that the error at the vertices is ~ 0 to machine precision (double precision, ~ 15 decimals). This is done by “inventing” a solution $u_D = 1 + x^2 + 2y^2$ and then calculating

$$-\nabla^2 u_D = -\nabla \cdot \nabla u_D = -\nabla \cdot \begin{pmatrix} 2x \\ 4y \end{pmatrix} = -(2 + 4) = -6 = f$$

Choosing $f = -6$ then result in the solution $u_D = 1 + x^2 + 2y^2$ if the Dirichlet boundary conditions are consistent with this u_D . This is a clever way of testing the implementation 😊

The FEM solution will match exactly at the nodes, but not within each triangle in the mesh, as u_D is a quadratic function and we are trying to approximate it with piecewise linear functions, so the L^2 error is “large”.

Try now to change the function space V to `FunctionSpace(mesh, 'P', 2)`. What happens and why?

Index notation

The governing equations of elasticity theory (and many other subjects) are often written using index notation. Here is a short guide to index notation:

1. Index notation write equations concerning the components of vector and tensors.
2. The components are accessed using an index, e.g., x_1 is the first component of a vector $\vec{x} = (x_1, x_2, x_3)^T$.
(The overhead arrow of a vector is often not written in this course...)
3. Accessing index using a letter, corresponds to accessing an arbitrary component, e.g., x_i ($i=1, 2$ or 3).
4. Matrix and tensor components are accessed using multiple indices, e.g., M_{ij} is any component of the matrix

$$M = \begin{bmatrix} M_{11} & M_{12} & M_{13} \\ M_{21} & M_{22} & M_{23} \\ M_{31} & M_{32} & M_{33} \end{bmatrix}$$

5. Repeated indices are summed over, but the sum is not written explicitly (Einstein's summation convention). For example

$$x_i x_i = \sum_i x_i^2 = \vec{x} \cdot \vec{x}$$

$$M_{ij} x_j = \sum_j M_{ij} x_j = (Mx)_i = i\text{'th component of the matrix-vector product } Mx$$

6. Indices not summed over are named free indices. Free indices should match on the left and right hands sides of an equation.
7. The Kronecker-delta symbol is δ_{ij} and is defined as

$$\delta_{ij} = \begin{cases} 1, & \text{if } i = j \\ 0, & \text{otherwise} \end{cases}$$

8. δ_{ij} substitutes indices, e.g., $\sigma_{ij} \delta_{jk} = \sigma_{ik}$.
9. A comma , means partial derivative, e.g.

$$v_{i,j} = \frac{\partial v_i}{\partial x_j} \text{ and } v_{i,i} = \sum_i \frac{\partial v_i}{\partial x_i} = \nabla \cdot \vec{v} \text{ and } \sigma_{ij,j} = \sum_j \frac{\partial \sigma_{ij}}{\partial x_j} = \nabla \cdot \sigma$$

Linear Elasticity

In linear elasticity theory, the deformation of a solid body Ω of a linear elastic isotropic material is governed by the following PDE:

$$-\nabla \cdot \sigma = f \text{ in } \Omega \quad , \quad \sigma = \lambda \text{tr}(\epsilon) I + 2\mu\epsilon \quad , \quad \epsilon = \frac{1}{2} (\nabla u + (\nabla u)^T)$$

1. u is the displacement vector field.
2. f is the body force density (force per unit volume) vector field.
3. σ is the stress tensor.
4. ϵ is the strain tensor.
5. I is the identity tensor.
6. λ and μ are Lamé parameters, related to the more common elastic constants as e.g.
 - a. $\lambda = 2K - \frac{2}{3}G$ and $\mu = G$. G is the shear modulus and K is the bulk modulus.
 - b. $\lambda = \frac{E\nu}{(1+\nu)(1-2\nu)}$ and $\mu = \frac{E}{2(1+\nu)}$. E is the elastic modulus and ν is Poisson's ratio. Valid in 3D and 2D plane strain.
 - c. $\lambda = \frac{E\nu}{1-\nu^2}$ and $\mu = \frac{E}{2(1+\nu)}$. E is the elastic modulus and ν is Poisson's ratio. Valid in 2D plane stress.

The equations can also be written using index notation:

$$-\sigma_{ij,j} = f_i \text{ in } \Omega \quad , \quad \sigma_{ij} = \lambda\epsilon_{kk}\delta_{ij} + 2\mu\epsilon_{ij} \quad , \quad \epsilon_{ij} = \frac{1}{2} (u_{i,j} + u_{j,i})$$

Using 6b. in the middle equation we get

$$\sigma_{ij} = \frac{E\nu}{(1+\nu)(1-2\nu)}\epsilon_{kk}\delta_{ij} + \frac{E}{(1+\nu)}\epsilon_{ij} = \frac{E}{1+\nu} \left(\epsilon_{ij} + \frac{\nu}{1-2\nu}\delta_{ij}\epsilon_{kk} \right)$$

This is the familiar generalized Hooke's law for a linear elastic isotropic material.

Linear Elasticity

In linear elasticity theory, the deformation of a solid body Ω of a linear elastic isotropic material is governed by the following PDE:

$$-\nabla \cdot \sigma = f \text{ in } \Omega \quad , \quad \sigma = \lambda \text{tr}(\epsilon) I + 2\mu\epsilon \quad , \quad \epsilon = \frac{1}{2} \left(\nabla u + (\nabla u)^T \right)$$

We want to solve this PDE for the displacement vector field u with some appropriate boundary conditions, to be discussed later.

To derive the weak form, we take the dot-product with a vector test function v and integrate over the body Ω :

$$\begin{aligned} \int_{\Omega} (-\nabla \cdot \sigma) \cdot v \, dx &= \int_{\Omega} f \cdot v \, dx \quad \Rightarrow \quad \int_{\Omega} \sigma : \nabla v \, dx - \int_{\partial\Omega} (\sigma \cdot \hat{n}) \cdot v \, dS = \int_{\Omega} f \cdot v \, dx \quad \Rightarrow \\ \int_{\Omega} \sigma : \nabla v \, dx &= \int_{\Omega} f \cdot v \, dx + \int_{\partial\Omega} (\sigma \cdot \hat{n}) \cdot v \, dS \quad \Rightarrow \quad \int_{\Omega} \sigma : \nabla v \, dx = \int_{\Omega} f \cdot v \, dx + \int_{\partial\Omega} T \cdot v \, dS \end{aligned}$$

Here partial integration was performed using the divergence theorem.

The $\sigma : \nabla v$ term is the sum of all pair-wise products of the elements of the two 2nd order tensors σ and ∇v .

$T = \sigma \cdot \hat{n}$ is named a surface tranction and is specified as a Neumann boundary condition.

A derivation using index notation (with implicit sums over repeated indices) reveals more clearly the forms of the terms:

$$\begin{aligned} -\sigma_{ij,j} = f_i \quad \Rightarrow \quad \int_{\Omega} (-\sigma_{ij,j} v_i) \, dx &= \int_{\Omega} f_i v_i \, dx \quad \Rightarrow \quad \int_{\Omega} \sigma_{ij} v_{i,j} \, dx - \int_{\partial\Omega} \sigma_{ij} \hat{n}_j v_i \, dS = \int_{\Omega} f_i v_i \, dx \quad \Rightarrow \\ \int_{\Omega} \sigma_{ij} v_{i,j} \, dx &= \int_{\Omega} f_i v_i \, dx + \int_{\partial\Omega} \sigma_{ij} \hat{n}_j v_i \, dS \quad \Rightarrow \quad \int_{\Omega} \sigma_{ij} v_{i,j} \, dx = \int_{\Omega} f_i v_i \, dx + \int_{\partial\Omega} T_i v_i \, dS \end{aligned}$$

where $T_i = \sigma_{ij} \hat{n}_j$ is named the surface traction.

Linear Elasticity

FEniCS actually support both index and vector notations. We will here stick with vector notation to follow the book. Keep in mind that e.g. $T \cdot v = T_i v_i$ and $\sigma : \nabla v = \sigma_{ij} v_{i,j}$ with implicit sums over repeated indices.

The weak form is: Find $u \in V$ such that

$$\int_{\Omega} \sigma : \nabla v \, dx = \int_{\Omega} f \cdot v \, dx + \int_{\partial\Omega_T} T \cdot v \, dS \quad \forall v \in \hat{V}$$

where $u = u_D$ and $v = 0$ is specified on the Dirichlet boundary $\partial\Omega_D$ and $T = \sigma \cdot \hat{n}$ is specified on the traction boundary $\partial\Omega_T$ and $\partial\Omega = \partial\Omega_D \cup \partial\Omega_T$.

We see that the Dirichlet boundary conditions corresponding to fixing the displacement.

The Neumann traction boundary conditions correspond to applying a surface traction to the boundary.

Since σ is symmetric and the product of a symmetric and an anti-symmetric tensor vanishes, we can also write

$$\sigma : \nabla v = \sigma : \left(\frac{1}{2} \left(\nabla v + (\nabla v)^T \right) + \frac{1}{2} \left(\nabla v - (\nabla v)^T \right) \right) = \sigma : \left(\frac{1}{2} \left(\nabla v + (\nabla v)^T \right) \right) + \sigma : \left(\frac{1}{2} \left(\nabla v - (\nabla v)^T \right) \right) = \sigma : \epsilon(v)$$

This is named the test strain or virtual strain.

The problem can now be written as: Find $u \in V$ such that $a(u, v) = L(v) \, \forall v \in \hat{V}$

$$a(u, v) = \int_{\Omega} \sigma(u) : \epsilon(v) \, dx \quad , \quad \sigma(u) = \lambda \text{tr}(\epsilon(u)) I + 2\mu \epsilon(u) \quad , \quad \epsilon(u) = \frac{1}{2} \left(\nabla u + (\nabla u)^T \right) \quad , \quad L(v) = \int_{\Omega} f \cdot v \, dx + \int_{\partial\Omega_T} T \cdot v \, dS$$

```
from fenics import *
```

```
# Variables
L = 1                # Length, [m]
W = 0.25             # Width, [m]
g = 9.81             # Gravitational acceleration, [m/s^2]

# PVC
E = 3e9              # Youngs modulus, [Pa]
nu = 0.38            # Poissons ration, [1]
mu = E/(2*(1+nu))    # Shear modulus, [Pa]
lambda_ = E*nu/((1+nu)*(1-2*nu)) # lambda is a reserved word in python
rho = 1380           # Density, [kg/m^3]
```

```
# Create mesh and define function space
mesh = BoxMesh.create([Point(0, 0, 0), Point(L, W, W)],
                      [50,10,10], CellType.Type.hexahedron)
V = VectorFunctionSpace(mesh, 'P', 1)
```

```
# Define boundary condition
tol = 1E-10
```

```
def clamped_boundary(x, on_boundary):
    # Return true if x=[x1,x2,x3] is on the clamped boundary
    return on_boundary and x[0] < tol
```

```
bc = DirichletBC(V, Constant((0, 0, 0)), clamped_boundary)
```

```
# Define strain and stress
```

```
def epsilon(u):
    return 0.5*(grad(u)+grad(u).T)
```

```
def sigma(u):
    eps=epsilon(u)
    return lambda_*tr(eps)*Identity(d)+2*mu*eps
```

```
# Define variational problem
u = TrialFunction(V)
d = u.geometric_dimension()
v = TestFunction(V)
f = Constant((0, 0, -rho*g))
T = Constant((0, 0, 0))
a = inner(sigma(u), epsilon(v))*dx
L = dot(f, v)*dx + dot(T, v)*ds
```

```
# Print number of unknowns
print('DOFs: %i'%V.dim())
```

```
# Compute solution
u = Function(V,name='Displacement') # 'name' shown in paraview
solve(a == L, u, bc)
```

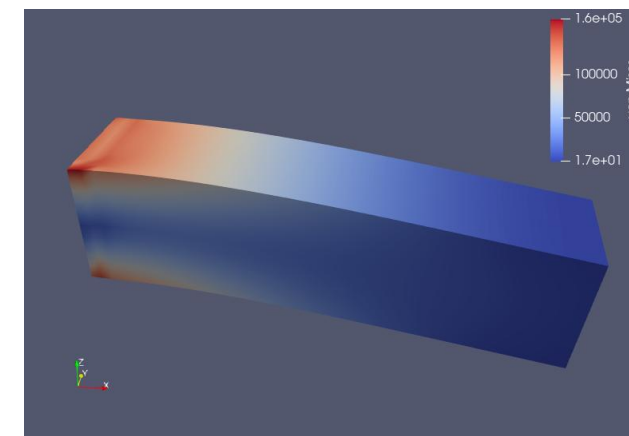
```
# Calculate stress
s = sigma(u) - (1./3)*tr(sigma(u))*Identity(d) # deviatoric stress
von_Mises = sqrt(3./2*inner(s, s))
V = FunctionSpace(mesh, 'P', 1)
von_Mises = project(von_Mises, V)
# The first string is the 'name'
von_Mises.rename('von Mises','von Mises stress')
```

```
# Calculate magnitude of displacement
u_magnitude = sqrt(dot(u, u))
u_magnitude = project(u_magnitude, V)
u_magnitude.rename('magnitude','magnitude')
print('min/max u:',
      u_magnitude.vector().min(),
      u_magnitude.vector().max())
```

```
# Save solution to file in VTK format
File('elasticity_modified/displacement.pvd') << u
File('elasticity_modified/von_mises.pvd') << von_Mises
File('elasticity_modified/magnitude.pvd') << u_magnitude
```

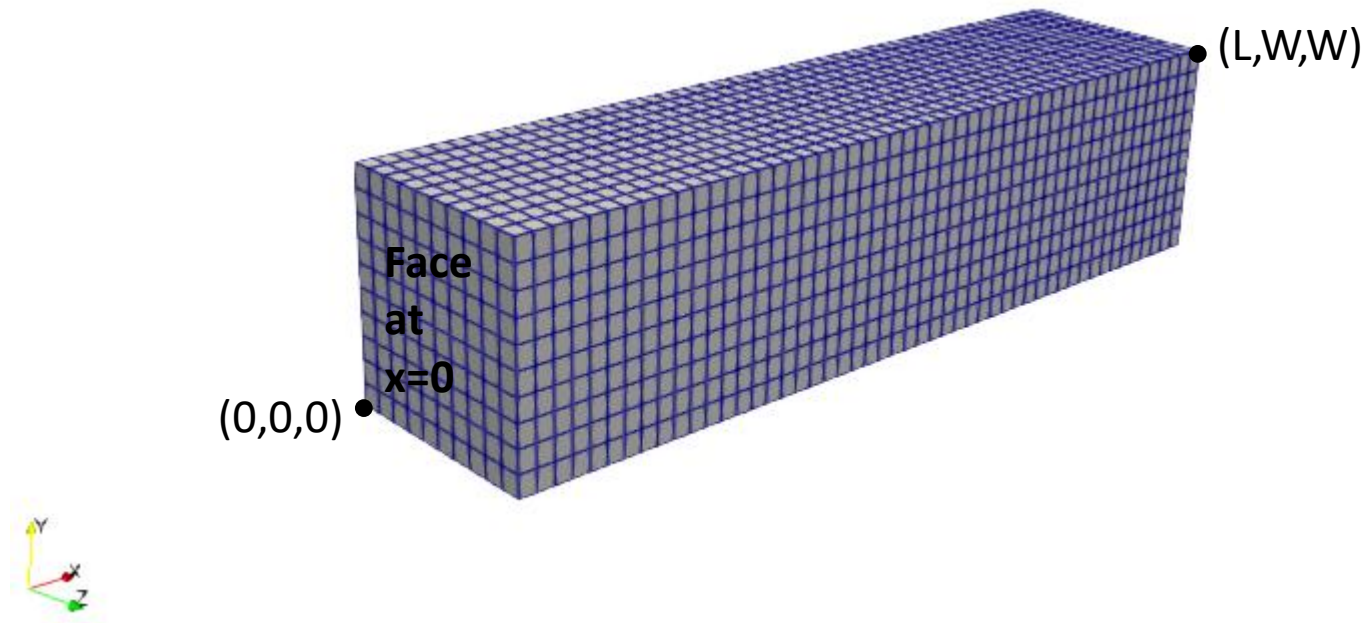
elasticity_modified.py

Beam under gravitational load



Part of code used to "place" the fixed boundary condition on the left end of the beam:

```
mesh = BoxMesh.create([Point(0, 0, 0), Point(L, W, W)],  
                      [50,10,10], CellType.Type.hexahedron)  
  
...  
# Define boundary condition  
tol = 1E-10  
  
def clamped_boundary(x, on_boundary):  
    # Return true if x=[x1,x2,x3] is on the clamped boundary  
    return on_boundary and x[0] < tol  
  
bc = DirichletBC(V, Constant((0, 0, 0)), clamped_boundary)
```



```
from fenics import *
```

```
# Variables
rho = 1380          # Density, [kg/m^3]
L = 1               # Length, [m]
W = 0.1             # Width, [m]
g = 9.81            # Gravitational acceleration, [m/s^2]
F = -rho*L*W*W*g    # Total force, [N]
traction = F/(W*W)  # Traction, [N/m^2]

# PVC
E = 3e9             # Youngs modulus, [Pa]
nu = 0.38           # Poissons ration, [1]
mu = E/(2*(1+nu))   # Shear modulus, [Pa]
lambda_ = E*nu/((1+nu)*(1-2*nu)) # lambda is a reserved word in python
```

```
I = W**4/12        # Moment of inertia
print('Euler beam max deflection: ',abs(F*L**3/3/E/I))
```

```
# Create mesh and define function space
mesh = BoxMesh.create([Point(0, 0, 0), Point(L, W, W)],
                      [50,10,10], CellType.Type.hexahedron)
```

```
# Define boundary condition
tol = 1E-10
```

```
class clamped_boundary(SubDomain):
    def inside(self, x, on_boundary):
        # Return true if x is on the clamped boundary
        return on_boundary and x[0] < tol
```

```
class load_boundary(SubDomain):
    def inside(self, x, on_boundary):
        # Return true if x is on the load boundary
        return on_boundary and abs(x[0]-L)<tol
```

```
boundary_parts = MeshFunction("size_t", mesh, 2) # 2 => 2D (surfaces)
boundary_parts.set_all(0)
load_boundary().mark(boundary_parts,1) # Load on boundary 1
clamped_boundary().mark(boundary_parts,2) # Fixed on boundary 2
```

```
# Save markings for inspection in paraview
File('elasticity_traction/boundary_parts.pvd') << boundary_parts
```

```
# Define new surface measure with marked surfaces
ds = Measure('ds', domain=mesh, subdomain_data=boundary_parts)
bc = DirichletBC(V, Constant((0, 0, 0)), boundary_parts, 2)
```

```
# Define strain and stress
```

```
def epsilon(u):
    return 0.5*(grad(u)+grad(u).T)
```

```
def sigma(u):
    eps=epsilon(u)
    return lambda_*tr(eps)*Identity(d)+2*mu*eps
```

```
# Define variational problem
u = TrialFunction(V)
d = u.geometric_dimension() # space dimension
v = TestFunction(V)
f = Constant((0, 0, 0)) # No body force
T = Constant((0, 0, traction))
a = inner(sigma(u), epsilon(v))*dx
L = dot(f, v)*dx + dot(T, v)*ds(1) # Traction only on boundary 1
```

```
# Print number of unknowns in equation system
print('DOFs: %i'%V.dim())
```

```
# Compute solution
u = Function(V,name='Displacement') # 'name' shown in paraview
solve(a == L, u, bc)
```

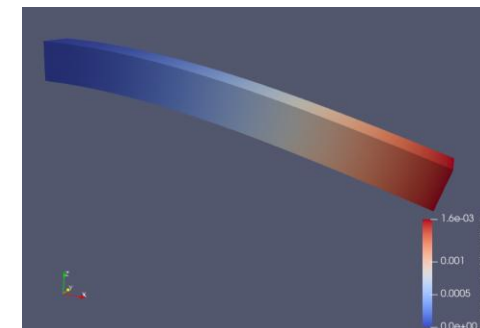
```
# Calculate stress
s = sigma(u) - (1./3)*tr(sigma(u))*Identity(d) # deviatoric stress
von_Mises = sqrt(3./2*inner(s, s))
V = FunctionSpace(mesh, 'P', 1)
von_Mises = project(von_Mises, V)
# The first string is the name (shown in paraview),
# the second is a description.
von_Mises.rename('von Mises','von Mises stress')
```

```
# Calculate magnitude of displacement
u_magnitude = sqrt(dot(u, u))
u_magnitude = project(u_magnitude, V)
u_magnitude.rename('magnitude','magnitude')
print('min/max u:',
      u_magnitude.vector().min(),
      u_magnitude.vector().max())
```

```
# Save solution to file in VTK format
File('elasticity_traction/displacement.pvd') << u
File('elasticity_traction/von_mises.pvd') << von_Mises
File('elasticity_traction/magnitude.pvd') << u_magnitude
```

elasticity_traction.py

Beam loaded on right surface



Part of code used to "mark" the boundary:

```
# Create mesh and define function space
mesh = BoxMesh.create([Point(0, 0, 0), Point(L, W, W)],
                      [50,10,10], CellType.Type.hexahedron)

# Define boundary condition
tol = 1E-10

class clamped_boundary(SubDomain):
    def inside(self, x, on_boundary):
        # Return true if x is on the clamped boundary
        return on_boundary and x[0] < tol

class load_boundary(SubDomain):
    def inside(self, x, on_boundary):
        # Return true if x is on the load boundary
        return on_boundary and abs(x[0]-L)<tol

boundary_parts = MeshFunction("size_t", mesh, 2) # 2 => 2D (surfaces)
boundary_parts.set_all(0)
load_boundary().mark(boundary_parts,1) # Load on boundary 1
clamped_boundary().mark(boundary_parts,2) # Fixed on boundary 2

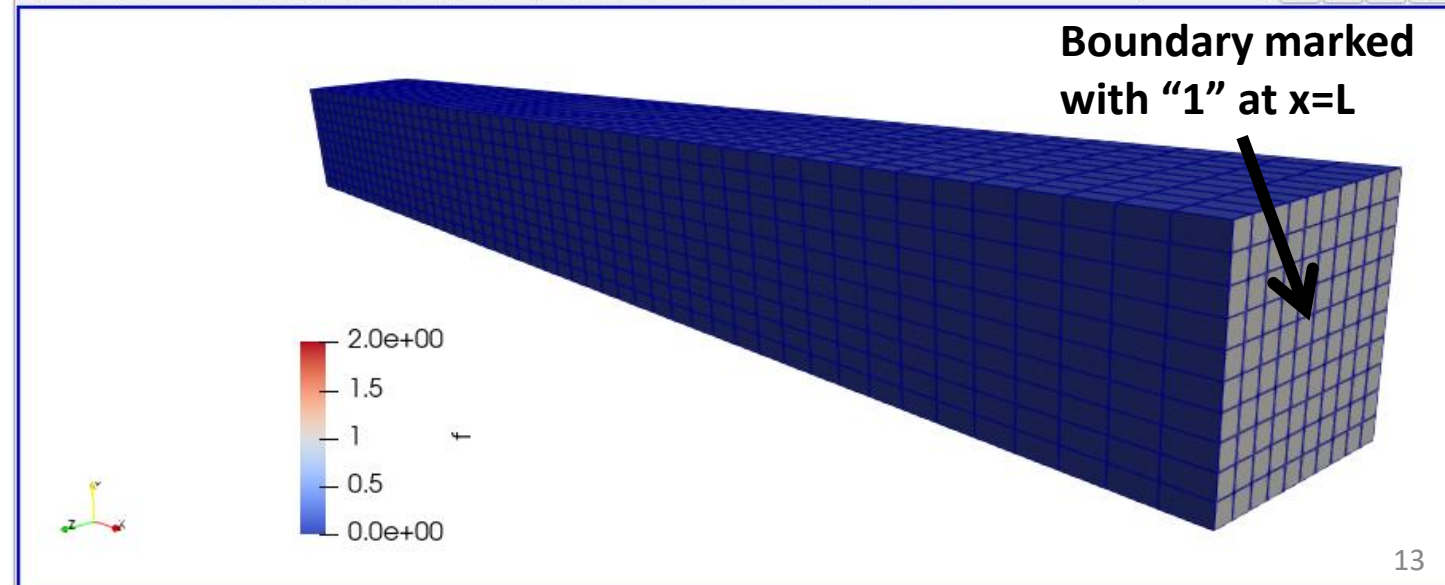
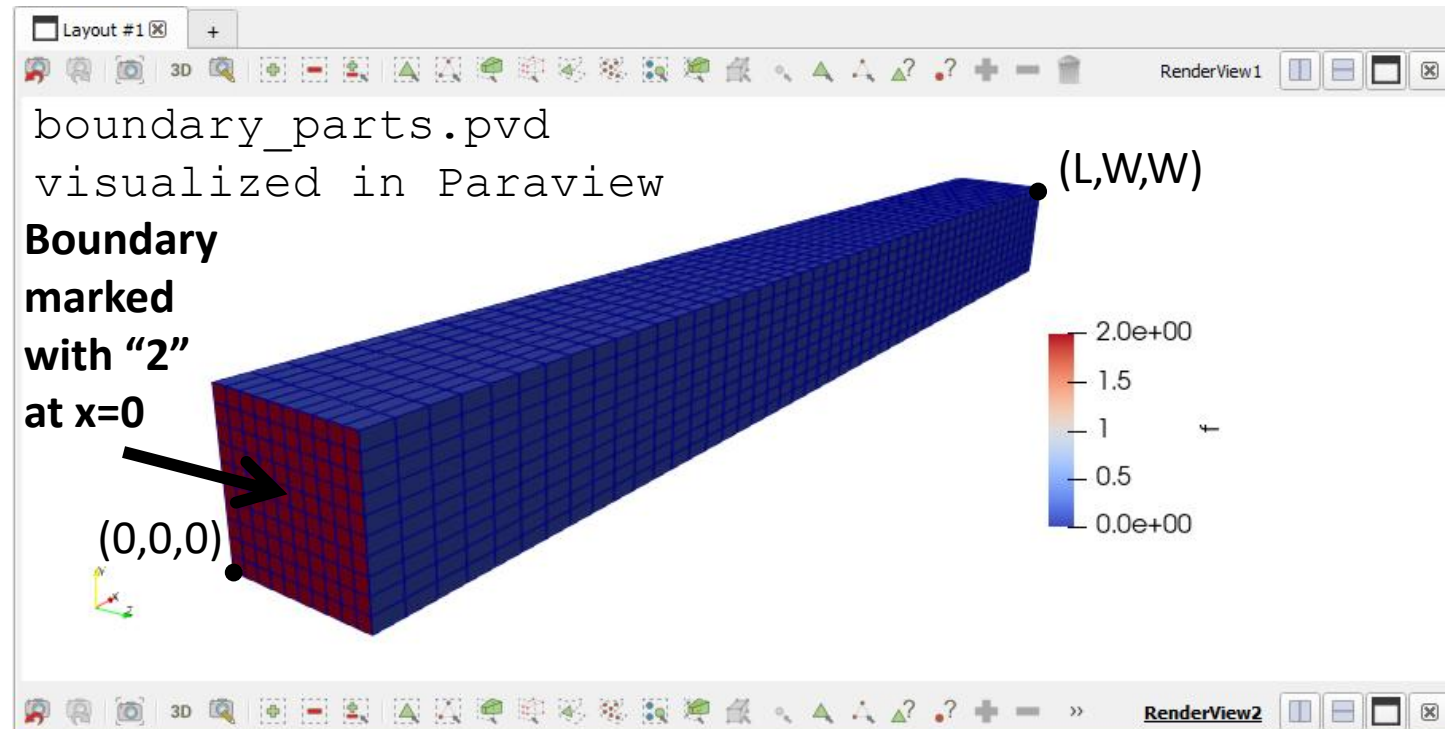
# Save markings for inspection in paraview
File('elasticity_traction/boundary_parts.pvd') << boundary_parts
```

Part of code used to "place" the fixed boundary condition on the x=0 end of the beam:

```
bc = DirichletBC(V, Constant((0, 0, 0)), boundary_parts, 2)
```

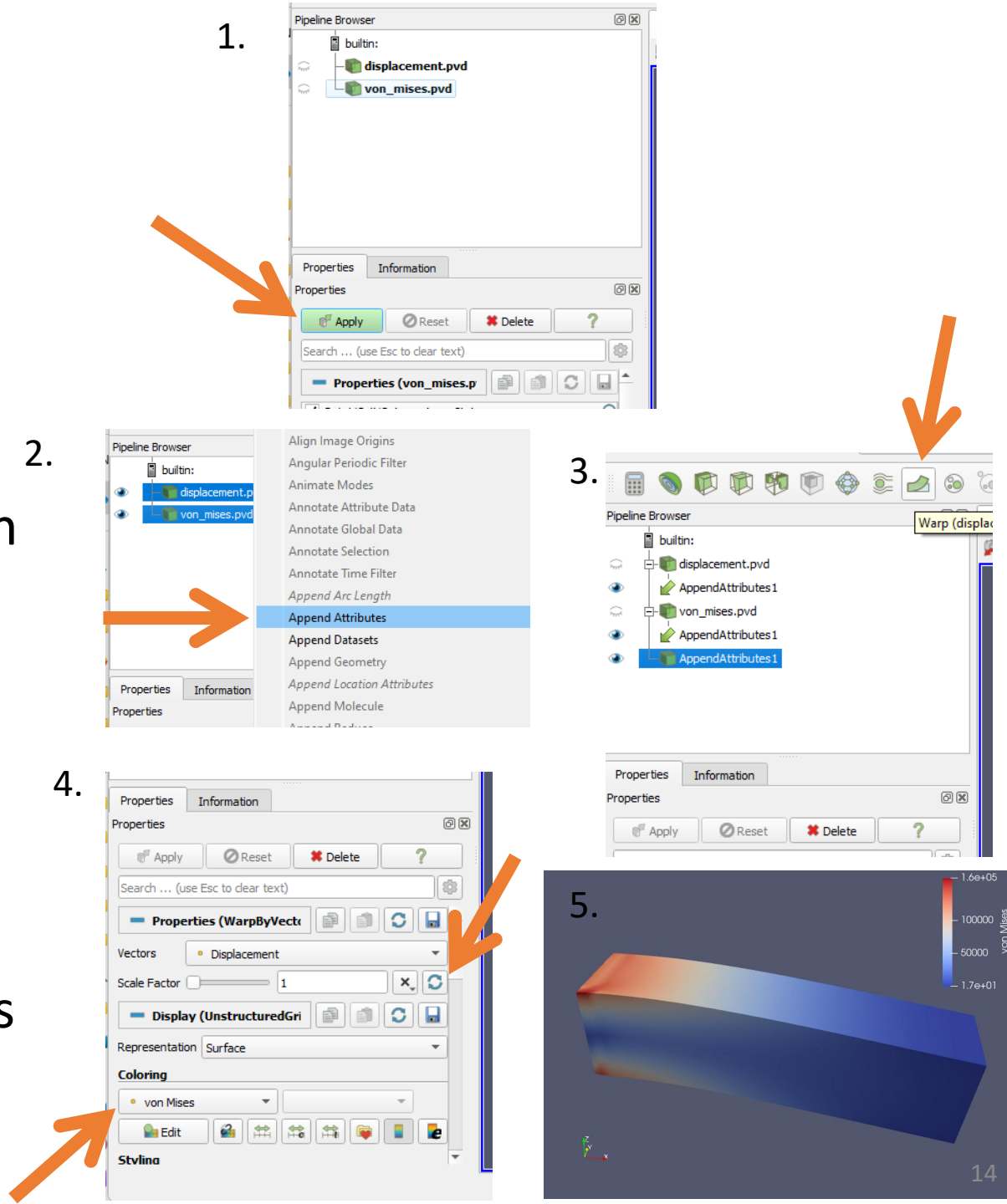
Part of code used to "place" the force at the x=L end of the beam:

```
ds = Measure('ds', domain=mesh, subdomain_data=boundary_parts)
...
L = dot(f, v)*dx + dot(T, v)*ds(1) # Traction only on boundary 1
```



Visualization in Paraview

1. Load data: File->Open find and select both 'displacement.pvd' and 'von_mises.pvd'. (press <ctrl> to select multiple files). Press OK and then press 'Apply'.
2. Select 'displacement.pvd' and 'von_mises.pvd' in the pipeline and select in the menu Filters->Alphabetical->Append Attributes. Press 'Apply'. Filters->Search can be useful if Alphabetical menu is too big for your screen! Append Attributes merges the data sets.
3. Select the last 'AppendAttributes1' in the pipeline and find 'Warp by vector' in filters or using the icon. Press 'Apply'.
4. Select 'Coloring' to be 'von Mises' and press the auto scale button . Press 'Apply'.
5. Enjoy the view!



Generalizing the formalism

The Poisson problem was previously formulated in the weak form as: Find $u \in T$ such that

$$\int_{\Omega} \nabla u \cdot \nabla v \, dV = \int_{\partial\Omega_N} gv \, dS + \int_{\Omega} v \frac{\rho}{\epsilon} \, dV \quad , \quad \forall v \in \hat{T}$$

A more generic form would be: Find $u \in T$ such that

$$a(u, v) = L(v) \quad , \quad \forall v \in \hat{T}$$

$a(u, v)$ is termed a 'bi-linear form' and $L(v)$ is a 'linear form'.

For the Poisson problem, $a(u, v)$ is the integral on the LHS of the weak form and $L(v)$ is the integral on the RHS of the weak form.

For other problems, $a(u, v)$ and $L(v)$ will have different expressions, but all linear problems can be written on the generic form above.

An even more general form of a problem would simply be: Find $u \in T$ such that

$$F(u, v) = 0 \quad , \quad \forall v \in \hat{T}$$

where $F(u, v)$ is some possible non-linear functional of u and a linear function of v .

Fenics 'lhs' and 'rhs' functions can be used to define a and L from F .

Non-linear problems

- If the PDE is non-linear, then we can apply the discretization method as usual.
- The resulting system of n equations with n unknowns will be non-linear.
- This non-linear system of equations must be solved with an appropriate method for solving systems of non-linear equations.
- One very popular method is Newton's method.
- We will now look at some of the details of Newton's method:
 - We can do it on the “algebraic level”, as described on this and the next few slides.
 - Or we can do it on the “PDE level”, as described later.

Newton's method at the algebraic level

- Newton's method can be used to solve the systems of non-linear equations, which come out of the Finite Element discretization procedure, for example:

$$\mathbf{K}(\mathbf{x})\mathbf{x} = \mathbf{L}(\mathbf{x}) \Rightarrow \mathbf{K}(\mathbf{x})\mathbf{x} - \mathbf{L}(\mathbf{x}) = \mathbf{0} = \mathbf{F}(\mathbf{x})$$

- We thus need to solve

$$\mathbf{F}(\mathbf{x}) = \begin{pmatrix} F_0(x_0, x_1, \dots, x_{n-1}) \\ F_1(x_0, x_1, \dots, x_{n-1}) \\ \vdots \\ F_{n-1}(x_0, x_1, \dots, x_{n-1}) \end{pmatrix} = \mathbf{0}$$

$\mathbf{x}$ is the vector of unknown (e.g., nodal values) and $\mathbf{F}(\mathbf{x})$ is the equation system where the mass matrix, stiffness matrix and load vector can depend on the unknowns in $\mathbf{x}$.

- A 1st order Taylor-expansion of each component around some vector $\mathbf{x}_i$ give

$$F_k(\mathbf{x}) = F_k(\mathbf{x}_i) + \nabla F_k(\mathbf{x}_i) \cdot (\mathbf{x} - \mathbf{x}_i) \quad , \quad k = 0, 1, \dots, n-1$$

- Collecting this into vector form give

$$\mathbf{F}(\mathbf{x}) = \mathbf{F}(\mathbf{x}_i) + \nabla \mathbf{F}(\mathbf{x}_i)(\mathbf{x} - \mathbf{x}_i)$$

where the matrix $\nabla \mathbf{F}(\mathbf{x}_i)$ has components $[\nabla \mathbf{F}(\mathbf{x}_i)]_{jk} = \frac{\partial F_j(\mathbf{x}_i)}{\partial x_k}$ and is named the Jacobian matrix of size $n \times n$.

Newton's method at the algebraic level

- We rename the matrix $\nabla F(\mathbf{x}_i)$ to $J(\mathbf{x}_i)$
- The equation $F(\mathbf{x}) = F(\mathbf{x}_i) + J(\mathbf{x}_i)(\mathbf{x} - \mathbf{x}_i) = \mathbf{0}$ is then solved for $\mathbf{x}$:

$$F(\mathbf{x}) = F(\mathbf{x}_i) + J(\mathbf{x}_i)(\mathbf{x} - \mathbf{x}_i) = \mathbf{0} \implies$$

$$J(\mathbf{x}_i)(\mathbf{x} - \mathbf{x}_i) = -F(\mathbf{x}_i) \implies$$

$$(\mathbf{x} - \mathbf{x}_i) = -J^{-1}(\mathbf{x}_i)F(\mathbf{x}_i) \implies$$

$$\mathbf{x} = \mathbf{x}_i - J^{-1}(\mathbf{x}_i)F(\mathbf{x}_i) = \mathbf{x}_{i+1}$$

- $\mathbf{x}_{i+1}$ is the values at the next iteration.
- $J^{-1}(\mathbf{x}_i)$ is the inverse Jacobian matrix evaluated at $\mathbf{x}_i$.
- An inverse matrix is used in each iteration:
 - **This should be avoided**, since it is in general computationally very expensive to compute inverse matrices.
 - **Instead, one should solve a set of linear equations** in each iteration, which is much quicker:
 - Obtain $(\mathbf{x} - \mathbf{x}_i) = \mathbf{d}_i$ by solving the linear equations $J(\mathbf{x}_i)\mathbf{d}_i = -F(\mathbf{x}_i)$ and set $\mathbf{x}_{i+1} = \mathbf{x}_i + \mathbf{d}_i$.

Newton's method at the algebraic level

- For stability, each iteration can be damped by a factor α , $0 < \alpha \leq 1$.
- One iteration is thus
 - Calculate the Jacobian matrix $J(\mathbf{x}_i)$ with components $\left[\frac{\partial F_j(\mathbf{x}_i)}{\partial x_k} \right]_{jk}$.
 - Solve the linear equations $J(\mathbf{x}_i)\mathbf{d}_i = -\mathbf{F}(\mathbf{x}_i)$.
 - Set $\mathbf{x}_{i+1} = \mathbf{x}_i + \alpha\mathbf{d}_i$.
- FEniCS 'solve' command works also for non-linear equations.
- One can define 'F' and then simply write
`solve(F==0,u,bc)`
- FEniCS then calculates the Jacobian of F and uses Newton's method to solve the non-linear equations.
 - In each iteration, as system of linear equations are solved.
 - The solution of non-linear problems is thus (a lot) harder than the solution of linear problems.

Newton's method on the PDE level

On the previous slide we applied Newton's method to the discrete algebraic equations.

Newton's method can also be applied to the non-linear variational problem Find $u \in T$ s.t.

$$F(u, v) = 0 \quad , \quad \forall v \in \hat{T}$$

In each iteration, the non-linear problem is linearized around the current guess at a solution, u_0 , and a linear problem is solved for the increment du .

To get Newton's methods if u was an ordinary variable, we would write

$$F(u, v) \approx F(u_0, v) + \left. \frac{\partial F(u, v)}{\partial u} \right|_{u=u_0} (u - u_0) = 0.$$

u is a function, which we think of as a vector, so we must instead use the directional derivative in the direction du defined as

$$\lim_{\epsilon \rightarrow 0} \frac{F(u_0 + \epsilon du, v) - F(u_0, v)}{\epsilon} = \left. \frac{dF(u_0 + \epsilon du, v)}{d\epsilon} \right|_{\epsilon=0} = a(du, v)$$

This is the change in F -value in the direction du divided by the step size in the limit of vanishing step size, $\epsilon \rightarrow 0$.

$a(du, v)$ is the continuous version of $\mathbf{J}(\mathbf{x}_i)\mathbf{d}_i$ on the previous slide.

The bi-linear form is $a(du, v)$ and the linear form is $l(v) = -F(u_0, v)$.

Newton's method solves in each iteration the problem: Find $du \in T$ such that

$$a(du, v) = l(v) \quad , \quad \forall v \in \hat{T}$$

and u_0 is updated as $u_0 = u_0 + \alpha du$ for the parameter α with $0 < \alpha \leq 1$. $\alpha = 1$ is Newton's method, $\alpha < 1$ is damped-Newton.

Non-linear Poisson equation

A coefficient $q(u)$ is inserted to make the equation non-linear

$$\begin{aligned} -\nabla \cdot (q(u) \nabla u) &= f && \text{in } \Omega \quad , \\ u &= u_0 && \text{on } \partial\Omega_D \subset \partial\Omega \quad , \\ \nabla u \cdot \hat{n} &= g && \text{on } \partial\Omega_N \subset \partial\Omega \quad , \quad \partial\Omega_N = \partial\Omega \setminus \partial\Omega_D \end{aligned}$$

Weak form (or variational form): Find $u \in T$ such that

$$\int_{\Omega} q(u) \nabla u \cdot \nabla v \, dx = \int_{\partial\Omega_N} g v \, dS + \int_{\Omega} v f \, dx \quad , \quad \forall v \in \hat{T}$$

It is seen that the "stiffness matrix term" now depends on the solution u , so the discretized equations become non-linear.

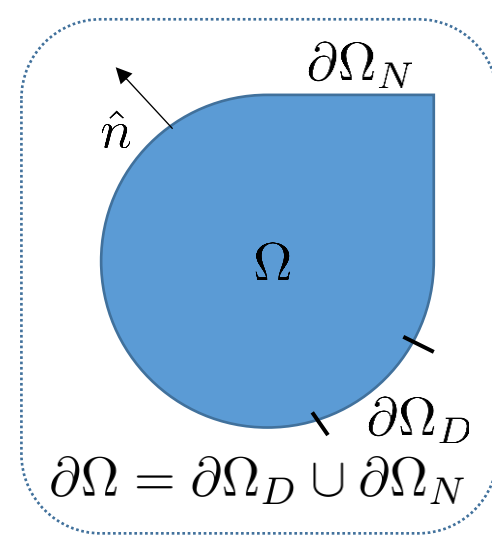
In the following fenics example we set $q(u) = 1 + u^2$ and consider only Dirichlet boundary conditions, i.e.

$$\begin{aligned} -\nabla \cdot ((1 + u^2) \nabla u) &= f && \text{in } \Omega \quad , \\ u &= u_0 && \text{on } \partial\Omega \end{aligned}$$

and then we get

$$F(u, v) = \int_{\Omega} (1 + u^2) \nabla u \cdot \nabla v \, dx - \int_{\Omega} v f \, dx = 0 \quad , \quad \forall v \in \hat{T}$$

For testing the implementation, the following example *chooses* $u_{\text{exact}} = 1 + x + 2y$ and then 'calculates backwards' to get $f(x, y) = -10x - 20y - 10$ so that the exact solution is known and we can test fenics solution against u_{exact} , since it should be found exactly with linear shape functions if the Dirichlet boundary conditions are set using the exact solution.



```

# Warning: from fenics import * will import both `sym`
# and `q` from FEniCS. We therefore import FEniCS
# first and then overwrite these objects.
from fenics import *
import matplotlib.pyplot as plt

def q(u):
    "Return nonlinear coefficient"#String used as comment...
    return 1 + u**2

# Use SymPy to compute f from the
# 'manufactured' solution u=1+x+2y
import sympy as sym
x, y = sym.symbols('x[0], x[1]')
u = 1 + x + 2*y
f = -sym.diff( q(u)*sym.diff(u,x) , x) - \
    sym.diff( q(u)*sym.diff(u,y) , y)
f = sym.simplify(f)
u_code = sym.printing.ccode(u) #Expression use C-code, which
f_code = sym.printing.ccode(f) #is also valid C++ code.
print('u =', u_code)
print('f =', f_code)

# Create mesh and define function space
mesh = UnitSquareMesh(8, 8)
V = FunctionSpace(mesh, 'P', 1)

# Define boundary condition from c-code from sympy
# calculation
u_D = Expression(u_code, degree=2)

def boundary(x, on_boundary):
    return on_boundary

bc = DirichletBC(V, u_D, boundary)

# Define variational problem
u = Function(V) # Note: not TrialFunction!
v = TestFunction(V)
f = Expression(f_code, degree=2) # Use the c-code
F = q(u)*dot(grad(u), grad(v))*dx - f*v*dx

# Compute solution
solve(F == 0, u, bc)

# Plot solution
plot(u)

# Compute maximum error at vertices. This
# computation illustrates
# an alternative to using compute_vertex_values as
# in poisson.py.
u_e = interpolate(u_D, V)
import numpy as np
error_max = np.abs(u_e.vector() - u.vector()).max()
print('error_max = ', error_max)

# Hold plot
plt.show()

```

Energy methods

Putting some sense into the test function...

The governing PDEs of a static physics problem often result from a minimization of a potential energy, such that we want to solve

$$\min_{u \in T} \Pi(u)$$

where Π is the potential energy.

The minimization problem is solved by finding a u for which a small variation of u leads to no change in $\Pi(u)$ to first order in any variation.

We change u to $u + \varepsilon v$ for some $\varepsilon \in \mathbb{R}$ and some test function $v \in \hat{T}$.

To minimize Π , we must solve the problem

$$F(u, v) = \left. \frac{d\Pi(u + \varepsilon v)}{d\varepsilon} \right|_{\varepsilon=0} = 0 \quad , \quad \forall v \in \hat{T}$$

where the derivative is named the Gateaux derivative and is the directional derivative of $\Pi(u)$ in the direction of v .

Note that if u is subject to Dirichlet boundary conditions, we do not want to consider variations that change the value of u at the Dirichlet boundary. This is accomplished by demanding that the test function v is zero at the Dirichlet boundary (as we also argued earlier).

Energy methods

The potential energy for a static mechanics problem is

Putting some sense into the test function...

$$\Pi = \Pi_{\text{internal}} + \Pi_{\text{external}}$$

$$\Pi_{\text{internal}}(u) = \text{Strain Energy} = \int_{\Omega} \frac{1}{2} \sigma_{ij} \epsilon_{ij} dV$$

$$\Pi_{\text{external}}(u) = \text{External energy} = - \int_{\Omega} f_i u_i dV - \int_{\partial\Omega_N} T_i u_i dS$$

where u_i is the displacement field, σ_{ij} is the stress tensor, ϵ_{ij} is the strain tensor, f_i is a body force and T_i is a surface traction.

Minimizing Π with respect to u_i gives a displacement field which satisfies the mechanical equilibrium equations.

Linear elasticity results from using the linearized strain tensor and a linear constitutive relation, e.g.

$$\sigma_{ij} = E_{ijkl} \epsilon_{kl} \quad , \quad \epsilon_{ij} = \frac{1}{2} (u_{i,j} + u_{j,i})$$

for a possible anisotropic material or with

$$E_{ijkl} = \frac{E}{1+\nu} \left(\frac{1}{2} (\delta_{ik} \delta_{jl} + \delta_{il} \delta_{jk}) + \frac{\nu}{1-2\nu} \delta_{ij} \delta_{kl} \right)$$

for an isotropic linear elastic material. The latter yield

$$\sigma_{ij} = 2\mu \epsilon_{ij} + \lambda \epsilon_{kk} \delta_{ij} = \mu (u_{i,j} + u_{j,i}) + \lambda u_{k,k} \delta_{ij} \quad , \quad \mu = \frac{E}{2(1+\nu)} \quad , \quad \lambda = \frac{E\nu}{(1+\nu)(1-2\nu)}$$

μ and λ are named Lamé parameters. μ is also the shear modulus.

Energy methods

$$F(u, v) = \left. \frac{d\Pi(u + \varepsilon v)}{d\varepsilon} \right|_{\varepsilon=0} = 0 \quad , \quad \forall v \in \hat{T}$$

The strain energy for linear elasticity is then

$$\Pi_{\text{internal}} = \int_{\Omega} \frac{1}{2} \sigma_{ij} \epsilon_{ij} \, dV = \int_{\Omega} \frac{1}{4} (\mu (u_{i,j} + u_{j,i}) + \lambda u_{k,k} \delta_{ij}) (u_{i,j} + u_{j,i}) \, dV$$

and

$$\begin{aligned} \Pi_{\text{internal}}(u + \varepsilon v) &= \int_{\Omega} \frac{1}{4} (\mu ((u + \varepsilon v)_{i,j} + (u + \varepsilon v)_{j,i}) + \lambda (u + \varepsilon v)_{k,k} \delta_{ij}) ((u + \varepsilon v)_{i,j} + (u + \varepsilon v)_{j,i}) \, dV \\ &= \int_{\Omega} \frac{1}{4} (\mu (u_{i,j} + u_{j,i}) + \lambda u_{k,k} \delta_{ij} + \varepsilon (\mu (v_{i,j} + v_{j,i}) + \lambda v_{k,k} \delta_{ij})) (u_{i,j} + u_{j,i} + \varepsilon (v_{i,j} + v_{j,i})) \, dV \\ \frac{d\Pi_{\text{internal}}(u + \varepsilon v)}{d\varepsilon} &= \int_{\Omega} \frac{1}{4} \left((\mu (v_{i,j} + v_{j,i}) + \lambda v_{k,k} \delta_{ij}) (u_{i,j} + u_{j,i} + \varepsilon (v_{i,j} + v_{j,i})) \right. \\ &\quad \left. + (\mu (u_{i,j} + u_{j,i}) + \lambda u_{k,k} \delta_{ij} + \varepsilon (\mu (v_{i,j} + v_{j,i}) + \lambda v_{k,k} \delta_{ij})) (v_{i,j} + v_{j,i}) \right) \, dV \\ \left. \frac{d\Pi_{\text{internal}}(u + \varepsilon v)}{d\varepsilon} \right|_{\varepsilon=0} &= \int_{\Omega} \frac{1}{4} \left((\mu (v_{i,j} + v_{j,i}) + \lambda v_{k,k} \delta_{ij}) (u_{i,j} + u_{j,i}) + (\mu (u_{i,j} + u_{j,i}) + \lambda u_{k,k} \delta_{ij}) (v_{i,j} + v_{j,i}) \right) \, dV \\ &= \int_{\Omega} \frac{1}{4} \left((2\mu (v_{i,j} + v_{j,i}) (u_{i,j} + u_{j,i}) + 2\lambda v_{k,k} u_{i,i}) \right) \, dV \\ &= \int_{\Omega} \sigma_{ij}(u) \epsilon_{ij}(v) \, dV \end{aligned}$$

where it was used that

$$\sigma_{ij}(u) = \mu (u_{i,j} + u_{j,i}) + \lambda u_{k,k} \delta_{ij} \quad , \quad \epsilon_{ij}(v) = \frac{1}{2} (v_{i,j} + v_{j,i})$$

Energy methods

The external energy gives

$$\begin{aligned}\Pi_{\text{external}} &= - \int_{\Omega} f_i u_i \, dV - \int_{\partial\Omega_N} T_i u_i \, dS \\ \left. \frac{d\Pi_{\text{external}}(u + \varepsilon v)}{d\varepsilon} \right|_{\varepsilon=0} &= - \int_{\Omega} f_i v_i \, dV - \int_{\partial\Omega_N} T_i v_i \, dS\end{aligned}$$

since the external energy is here linear in u_i .

The problem to be solved is: Find $u \in T$ such that

$$F(u, v) = \left. \frac{d\Pi(u + \varepsilon v)}{d\varepsilon} \right|_{\varepsilon=0} = 0 \quad , \quad \forall v \in \hat{T}$$

which boils down to: Find $u \in T$ such that

$$\begin{aligned}a(u, v) &= \int_{\Omega} \sigma_{ij}(u) \epsilon_{ij}(v) \, dV = l(v) = \int_{\Omega} f_i v_i \, dV + \int_{\partial\Omega_N} T_i v_i \, dS \quad , \quad \forall v \in \hat{T} \\ \sigma_{ij}(u) &= \mu (u_{i,j} + u_{j,i}) + \lambda u_{k,k} \delta_{ij} \quad , \quad \epsilon_{ij}(v) = \frac{1}{2} (v_{i,j} + v_{j,i})\end{aligned}$$

This is the same weak form as we saw earlier (but now in index notation), where we earlier started from the equilibrium equations.

The problem of linear elasticity can thus also be stated as a minimization problem.

Energy methods

Geometric non-linearity can be included by using the Green-Lagrange strain tensor instead of the linearized strain tensor

$$\epsilon_{ij} = \frac{1}{2} (u_{i,j} + u_{j,i} + u_{k,i}u_{k,j})$$

in the linear constitutive model, $\sigma_{ij} = 2\mu\epsilon_{ij} + \lambda\epsilon_{kk}\delta_{ij}$,

$$\Pi_{internal} = \int_{\Omega} \frac{1}{2} \sigma_{ij} \epsilon_{ij} \, dV = \int_{\Omega} \frac{1}{2} (2\mu\epsilon_{ij} + \lambda\epsilon_{kk}\delta_{ij}) \epsilon_{ij} \, dV = \int_{\Omega} \left(\mu\epsilon_{ij}\epsilon_{ij} + \frac{\lambda}{2}\epsilon_{ii}^2 \right) \, dV$$

which is known as the St. Venant-Kirchhoff model.

Calculating the Gateaux derivative and setting it to zero results in the weak form of the problem, which is now non-linear and generally written as $F(u, v) = 0$.

The problem can then be solved with e.g. a Newton-method, such that a linear problem is solved in each iteration towards the solution of the non-linear problem.

Fenics allows one to write the energy, and then symbolically take the Gateaux derivative to derive the weak form equations as $F(u, v) = 0$.

It will also symbolically calculate the Jacobian of these equations to be used with a Newton-method for solving the non-linear problem.

In this way, one just needs to define the energy and the software derives the weak form equations and the linearized problem together with all the FEM matrices and solves the whole non-linear problem using Newton's method.

$$\Pi = \Pi_{\text{internal}} + \Pi_{\text{external}} \quad , \quad \Pi_{\text{internal}}(u) = \int_{\Omega} \frac{1}{2} \sigma_{ij} \epsilon_{ij} dV = \int_{\Omega} \left(\mu \epsilon_{ij} \epsilon_{ij} + \frac{\lambda}{2} \epsilon_{ii}^2 \right) dV \quad , \quad \Pi_{\text{external}}(u) = - \int_{\Omega} f_i u_i dV - \int_{\partial\Omega_N} T_i u_i dS$$

The problem to be solved is: Find $u \in T$ such that

$$F(u, v) = \left. \frac{d\Pi(u + \varepsilon v)}{d\varepsilon} \right|_{\varepsilon=0} = 0 \quad , \quad \forall v \in \hat{T}$$

Part of the fenics implementation is shown below. The full code is available on Brightspace.

elasticity_traction_nonlinear.py

```
# PVC
E = 3e9                # Youngs modulus, [Pa]
nu = 0.38              # Poissons ration, [1]
mu = E/(2*(1+nu))      # Shear modulus, [Pa]
lambda_ = E*nu/((1+nu)*(1-2*nu))
# lambda is a reserved word in python

# Define variational problem
u = Function(V,name='Displacement')
du = TrialFunction(V)
v = TestFunction(V)
f = Constant((0, 0, 0))      # No body force
T = Constant((0, 0, traction))

# Linear strain
#eps = (grad(u)+grad(u).T)/2

# Green-Lagrange strain
eps = (grad(u)+grad(u).T+dot(grad(u),grad(u).T))/2

# Stored strain energy density
Psi = mu*tr(eps*eps)+lambda_/2*tr(eps)**2

# Total potential energy
Pi = Psi*dx - dot(f, u)*dx - dot(T, u)*ds(1)

# Directional derivative about u in the direction of v
# give the nonlinear equations F(u,v)=0
F = derivative(Pi, u, v)

# Compute Jacobian of F for Newton's method
dF = derivative(F, u, du)

# Create nonlinear variational problem and solve
problem = NonlinearVariationalProblem(F, u, bcs=bc, J=dF)
solver = NonlinearVariationalSolver(problem)
solver.parameters['newton_solver']['absolute_tolerance'] = 1e-6
solver.parameters['newton_solver']['linear_solver'] = 'cg'
solver.parameters['newton_solver']['preconditioner'] = 'sor'
solver.solve()
```

$$\Pi = \Pi_{\text{internal}} + \Pi_{\text{external}} \quad , \quad \Pi_{\text{internal}}(u) = \int_{\Omega} \frac{1}{2} \sigma_{ij} \epsilon_{ij} dV = \int_{\Omega} \left(\mu \epsilon_{ij} \epsilon_{ij} + \frac{\lambda}{2} \epsilon_{ii}^2 \right) dV \quad , \quad \Pi_{\text{external}}(u) = - \int_{\Omega} f_i u_i dV - \int_{\partial\Omega_N} T_i u_i dS$$

The problem to be solved is: Find $u \in T$ such that

$$F(u, v) = \left. \frac{d\Pi(u + \varepsilon v)}{d\varepsilon} \right|_{\varepsilon=0} = 0 \quad , \quad \forall v \in T$$

Part of the fenics implementation is shown below. The full code is available on Brightspace.

elasticity_traction_nonlinear.py

```
# PVC
E = 3e9          # Youngs modulus, [Pa]
nu = 0.38        # Poissons ration, [1]
mu = E/(2*(1+nu)) # Shear modulus, [Pa]
lambda_ = E*nu/((1+nu)*(1-2*nu))
# lambda is a reserved word in python

# Define variational problem
u = Function(V,name='Displacement')
du = TrialFunction(V)
v = TestFunction(V)
f = Constant((0, 0, 0))      # No body force
T = Constant((0, 0, traction))

# Linear strain
#eps = (grad(u)+grad(u).T)/2

# Green-Lagrange strain
eps = (grad(u)+grad(u).T+dot(grad(u),grad(u).T))/2

# Stored strain energy density
Psi = mu*tr(eps*eps)+lambda_/2*tr(eps)**2

# Total potential energy
Pi = Psi*dx - dot(f, u)*dx - dot(T, u)*ds(1)

# Directional derivative about u in the direction of v
# give the nonlinear equations F(u,v)=0
F = derivative(Pi, u, v)

# Compute Jacobian of F for Newton's method
dF = derivative(F, u, du)

# Create nonlinear variational problem and solve
problem = NonlinearVariationalProblem(F, u, bcs=bc, J=dF)
solver = NonlinearVariationalSolver(problem)
solver.parameters['newton_solver']['absolute_tolerance'] = 1e-6
solver.parameters['newton_solver']['linear_solver'] = 'cg'
solver.parameters['newton_solver']['preconditioner'] = 'sor'
solver.solve()
```

Linear strain

Non-linear strain

The deformation has NOT been scaled!

